From Digital Systems to Microcontrollers

Introduction to ARM Cortex-M Microprocessors — Chapter 1

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Embedded Systems
3. Inside the Computer
4. Buses and Memory-Mapped I/O
5. Inside the CPU
6. Execution Cycle and Program Trace
7. Meet the Board
8. From C to a Pin

Why This Chapter Matters

Your gate circuits did only what their wiring told them. To change the behaviour, you had to rewire. A microcontroller is a digital circuit too, but a program stored in memory controls it. The same silicon can blink an LED, measure a temperature, or drive a motor. It depends only on the bits you load.

After this chapter, you can:

1. Define an embedded system and its design constraints.
2. Distinguish a microprocessor from a microcontroller.
3. Name the CPU functional units and the role of each.
4. Explain the address, data and control buses, and compute addressable memory.
5. Trace a short program through fetch–decode–execute.
6. Describe how one C statement changes a voltage on a pin.

© Munim Zabidi

This chapter introduces many new terms. Every section answers just one question. This question is the thread that connects everything that follows:

> How does a program cause hardware to do something?

The one fact to carry out of this chapter

The CPU has no special instruction for “turn on an LED”.

It only has instructions that read and write memory addresses. Some of these addresses connect not to memory, but to hardware instead.

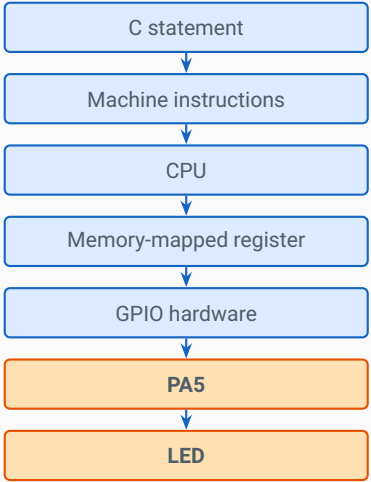
So turning on an LED means writing a number to an address.

© Munim Zabidi

The Destination

Before we learn the vocabulary, let us first see where we are going.

By the end of this chapter, you will be able to explain every arrow in this chain. You do not need to understand it yet.

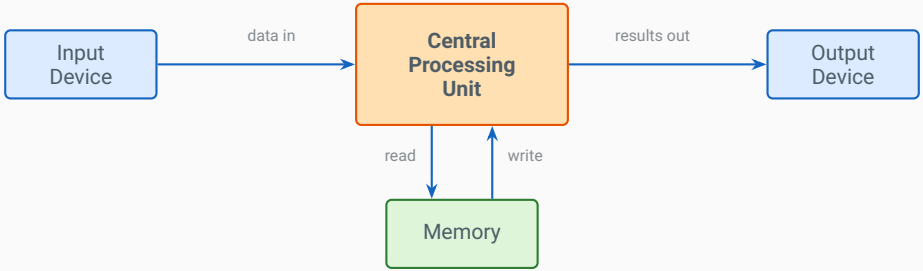


© Muním Zabidi

Table of Contents

1. Why This Matters
2. Embedded Systems
3. Inside the Computer
4. Buses and Memory-Mapped I/O
5. Inside the CPU
6. Execution Cycle and Program Trace
7. Meet the Board
8. From C to a Pin

Every stored-program computer, since the earliest ones, has had the same four components.

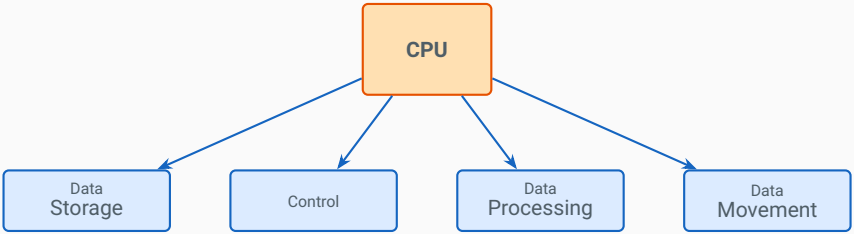


Computer

A complete system that processes, stores, and moves data, and exerts control, all under the guidance of a program. It has a processor, memory, and input/output (I/O) peripherals.

© Munim Zabidi

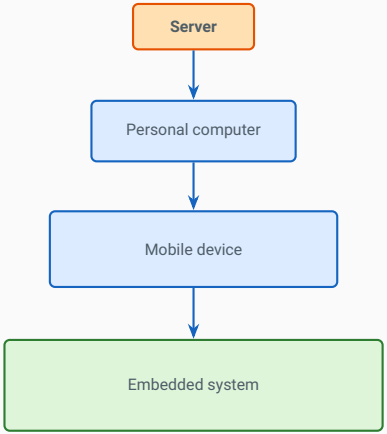
The Four Basic Functions of a Computer



- **Data Processing** — arithmetic and logical operations on data
- **Data Storage** — keeping program and data in memory, short- or long-term
- **Data Movement** — transferring data between CPU, memory, and I/O
- **Control** — sequencing and timing every other operation correctly

© Munim Zabidi

Classifying Computers by Numbers



- **Server** — fewest units; shared by thousands of users at once
- **Personal computer** — one per desk
- **Mobile device** — one or more per person
- **Embedded system** — dozens per person, hidden inside everyday products

Box size is the count, not the physical size

A server is physically the largest machine here, but there are the fewest of them. An embedded system is tiny, but outnumbers every other kind combined.

© Muním Zabidi

What Makes a System Embedded

Every machine on the last slide is still just a computer: processor, memory, I/O. The difference is what the processor is for. A general-purpose computer is built to run whatever the user installs next. An embedded system is built into a product to perform one job.

General-purpose computer

- Runs many different applications
- End-user installs new software freely
- Faster is always better
- Sold on speed, compatibility, price

Embedded system

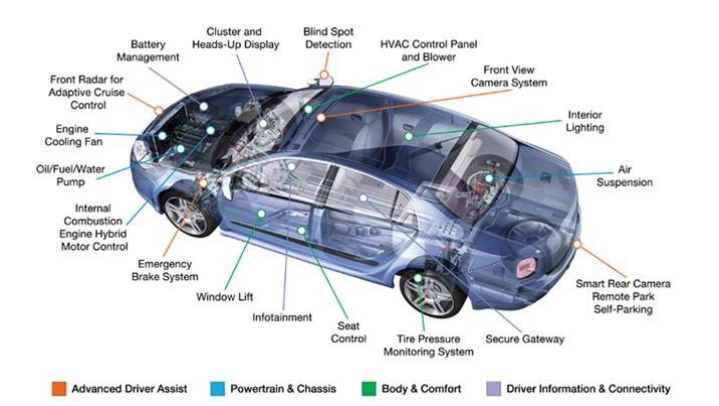
- Runs a few functions, fixed at design time
- Firmware is written once, by the designer
- Must meet a fixed deadline; extra speed may not help
- Sold on power, cost, size, predictability

The idea that matters

A desktop that answers in 100 ms instead of 10 ms is just slow. But an airbag controller that fires in 100 ms instead of 10 ms has failed.

Embedded engineering cares about *predictable* time, not maximum speed.

Embedded Systems: An Example



Each label is at least one ECU: an **Electronic Control Unit**. It is a micro-controller, its firmware, and a network connection, all together.

A car today carries **50–150** of these. By **2030**, everyday cars are expected to carry 200–250.

The driver never programs any of them directly.

Diagram credit: Embitel.

An ECU is classified by the function it controls: engine control module (ECM), body control module (BCM), electronic brake control module (EBCM), powertrain control module (PCM), transmission control module (TCM), suspension control module (SCM), door control unit (DCM), battery management system (BMS), and many more.

© Muním Zabidi

Constraint	Why it binds	What it forces
Size	Fits inside the product it controls	Package, pin count, board area
Weight	Portable, airborne, wearable	Battery size, materials
and		
Power	Battery life, thermal budget	Clock speed, sleep modes, duty cycle
Cost	Multiplied by production volume	Part choice, external components

Cost is multiplied, not added

Saving RM0.50 on one part looks trivial. But at 500,000 units, that saving becomes RM250,000. This is why embedded engineers argue about single resistors.

© Muním Zabidi

Table of Contents

1. Why This Matters
2. Embedded Systems
3. **Inside the Computer**
4. Buses and Memory-Mapped I/O
5. Inside the CPU
6. Execution Cycle and Program Trace
7. Meet the Board
8. From C to a Pin

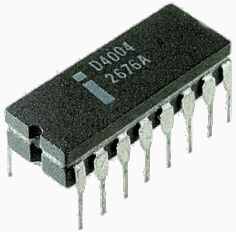
SWaP-C is a design constraint. Integrating the processor, memory, and peripherals onto one chip is how that constraint gets met in practice: fewer parts, smaller board, lower cost per unit. This single-chip design is called a **microcontroller**.

Feature	Microprocessor (μP)	Microcontroller (μC)
Integration	Processor chip only	Processor, RAM, ROM, I/O on one die
Flexibility	Memory and peripherals scale	Resources fixed inside the chip
Complexity	Many components, complex PCB	Few components, simple board
Power, cost	Higher	Optimised for low power and unit cost
Performance	High clock, heavy computation	Lower clock, control and I/O

A microprocessor needs help; a microcontroller comes ready

A microprocessor generally has no RAM, no ROM, and no way to reach the outside world on-chip; those parts must be wired around it before it can do anything. A microcontroller puts all of this together on one chip. The Nucleo's chip is a microcontroller: everything on the earlier board slide sits inside one package.

© Muním Zabidi



The first commercially produced microprocessor was the **Intel 4004** (1971) — a complete 4-bit CPU on one chip. Every processor in this book descends from that idea, scaled up over five decades.

Image: Wikimedia Commons.

© Munim Zabidi

Memory: ROM and RAM

ROM — non-volatile

- Contents survive with no power
- Holds the program: fixed at manufacture, or "burned" with a programmer
- **Flash** is the dominant type: NAND or NOR
- STM32 devices use NOR flash for code

RAM — volatile

- Contents lost without power
- Holds temporary data: variables, the stack
- **SRAM**: fast, every embedded system has it
- **DRAM**: cheaper, needs refresh; only larger systems bother

On the Nucleo-F446RE

512 KB Flash holds your program. **128 KB SRAM** holds your variables and stack. Chapter 3 returns to this split in detail.

© Muním Zabidi

Step 1 of fetch–decode–execute reads a “machine code instruction.” What is that, really?

Machine language

The lowest level. A bit pattern, such as `0001 1011`, that the Decoder is built to interpret. This is the only language the CPU itself understands.

Assembly language

A human-readable mnemonic per instruction — `MOV`, `ADD`, `MUL` — translated to machine code by an **assembler**. One line in, one instruction out.

A **cross assembler** runs on one computer (your PC) and generates machine code for a different one (the Cortex-M4) — the normal situation in embedded development.

Chapter 5 returns to assembly language in depth, once the Cortex-M4’s own registers and instructions are in place.

© Muním Zabidi

Zooming In: From System to Operation

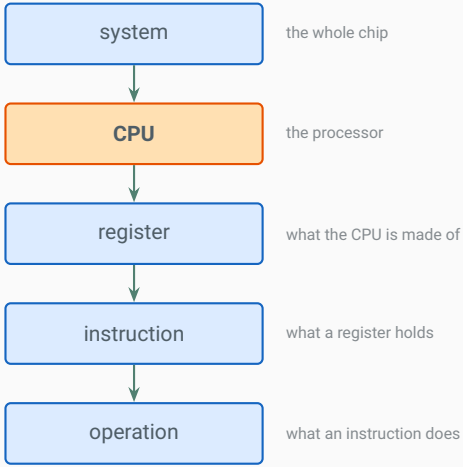
The next few slides introduce several names together: buses, registers, ALU, decoder. Each one belongs to one of two jobs: **moving** data from place to place, or **processing** it once it arrives.

Think of this as one picture, zoomed in one level at a time — top to bottom in the chain on the right.

Level 1: the whole microcontroller

CPU, RAM, Flash, GPIO, UART, ADC — all on one chip, connected by buses.

Buses move data; the CPU's internal parts process it. Buses let the CPU *reach* the rest of the chip. Registers are what the CPU is *made of*.



© Munim Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Inside the Computer

4 **Buses and Memory-Mapped I/O**

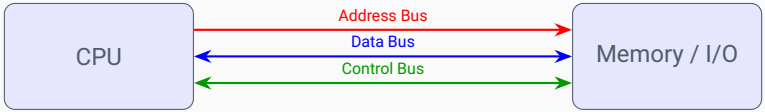
5 Inside the CPU

6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin

The Three Buses



Bus	Direction	Carries
Address	Unidirectional, CPU → device	*Which* location. Width sets the addressable range
Data	Bidirectional	*What* value. Width sets how much moves per transfer
Control	Bidirectional	*When* and *what kind*: read/write, timing, interrupts

Address width sets the memory map

2^n addressable bytes for an n-bit address bus.

$2^{16} = 64\,\text{KB}$ · $2^{20} = 1\,\text{MB}$ · $2^{32} = 4\,\text{GB}$: the Cortex-M4

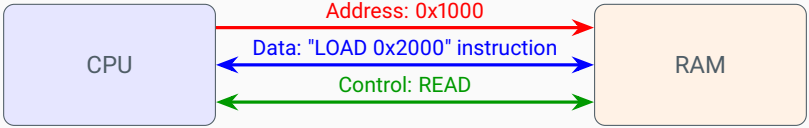
You already know this: an address is the input to a decoder. It works the same way as the one-of-*N* selection you built with a 3-to-8 decoder, just at a much larger scale.

© Muním Zabidi

Using the Buses: Fetching an Instruction

Suppose the CPU needs the instruction stored at `0x1000`:

Bus	Carries, this step
Address bus	`0x1000` — CPU asks for this location
Control bus	`READ` — CPU asserts a read
Memory	responds with `LOAD 0x2000`
Data bus	carries that instruction back to the CPU



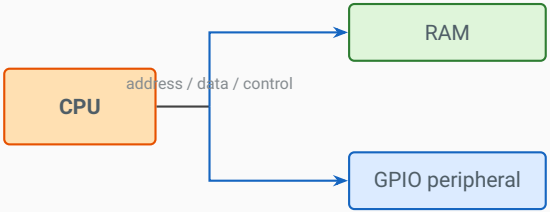
The same pattern repeats for every data access

The address bus names the location. The control bus says read or write. The data bus carries the value. Three buses, one transaction. We will use this exact pattern again when we trace a program.

© Munim Zabidi

Memory-Mapped I/O

The CPU talks to peripherals the same way it talks to memory: same address bus, same data bus, same read/write control line.



How it works

- Each peripheral register gets an address in the memory map.
- A read or write to that address reaches the register, not RAM — the peripheral answers.

Key idea: ordinary `LDR`/`STR` reach a peripheral register — the address alone decides who responds.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Inside the Computer

4 Buses and Memory-Mapped I/O

5 **Inside the CPU**

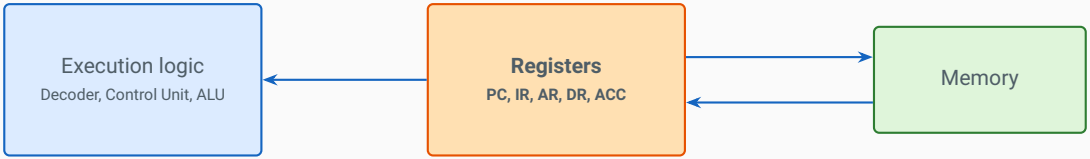
6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin

Two Halves: Execution Logic and Registers

Inside the CPU, hardware splits into two parts. Only one of them can reach memory directly.



The rule that shapes everything else

Only the registers touch memory. The decoder, control unit, and ALU have no direct memory access of their own — everything they need must first be fetched into a register.

© Muním Zabidi

The Execution Logic: Three Roles

Decoder

Reads the raw bit pattern of an instruction from the IR and works out what operation and what operands it names.

Control Unit

Generates the control signals that coordinate everything else: which register reads, which writes, when the ALU runs.

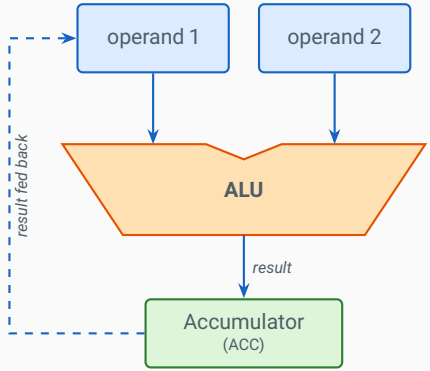
ALU

Does the arithmetic (add, subtract) and logic (AND, OR, XOR). Its bit width is usually called "the size of the processor."

What "32-bit" means here

4 bits was enough for a remote control. 8 served classic microcontrollers (8051, PIC). The Cortex-M4 is 32 bits — the mainstream embedded choice today. Servers run 64.

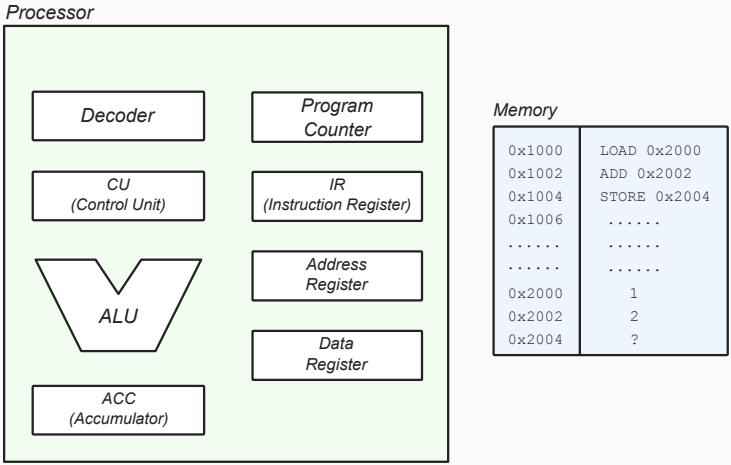
The Accumulator: Where Results Live



- The **ACC** is a key general-purpose register.
- Before an ALU operation, it holds an operand.
- After the operation, it receives the result.
- The dashed arrow shows why this matters: a result can become the next operand, without ever leaving the CPU.

© Munim Zabidi

Putting It Together: The Full Picture



Five registers, three execution units, one memory. The $z = x + y$ trace on the next slides puts every one of them to work.

Zoom 3: The Registers That Matter

Five names appear on the previous diagram. Not all of them need the same amount of attention yet:

Must understand now

- **PC** — address of the next instruction
- **IR** — the instruction just fetched
- **ACC** — operand and result of the ALU

Can recognise, needn't memorise yet

- **AR** — address of the data being accessed
- **DR** — value in transit to or from that address

The principle behind load/store architectures

The ALU operates on register operands only. Getting a value from memory into a register, or a result back out, takes a dedicated transfer. On the Cortex-M4 that is `LDR` and `STR`: `LDR` the operands in, `ADD` them in registers, `STR` the result back out. Chapters 5–7 explain this idea in more detail.

Chapter 2 replaces this generic register set with the Cortex-M4's actual sixteen registers. For now, just keep this simple picture in mind.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Inside the Computer

4 Buses and Memory-Mapped I/O

5 Inside the CPU

6 **Execution Cycle and Program Trace**

7 Meet the Board

8 From C to a Pin

Consider the simplest possible program:

```
z = x + y;
```

Assume the compiler places `x` at `0x2000`, `y` at `0x2002`, and `z` at `0x2004`.

One line of C becomes three machine instructions. Every register and operation from the last few slides gets a job to do. `LOAD`/`ADD`/`STORE` are teaching instructions for this generic model — Chapter 5 gives the real Cortex-M4 ones: `LDR`, `ADD`, `STR`.

Program memory

Address	Instruction
0x1000	LOAD 0x2000
0x1002	ADD 0x2002
0x1004	STORE 0x2004

Data memory

Address	Variable	Value
0x2000	x	1
0x2002	y	2
0x2004	z	?

© Muním Zabidi

Six Questions, Six Answers

Each new concept earns its place here by answering one question in this trace:

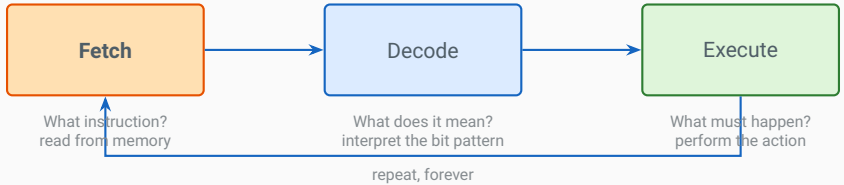
	Question	Answer
1	What is x?	value at `0x2000`
2	How does the CPU obtain it?	`LOAD`
3	Where does it go?	`ACC`
4	How does addition happen?	the `ALU`
5	Where does the result go?	`ACC`
6	How does it become z?	`STORE`

The next three slides execute this table, step by step

Each question above becomes one phase of fetch–decode–execute. Watch each register earn its answer.

© Muním Zabidi

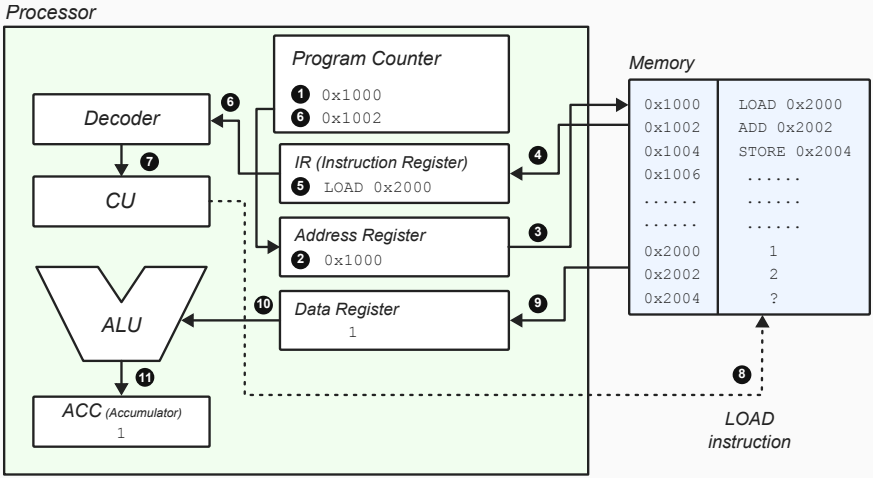
The CPU has fetched an instruction. What does it actually do with it?



This is a finite state machine, driven by the clock. You built FSMs in digital electronics. A CPU is the largest one you have met so far. The loop never stops while the machine has power. This is why an embedded program's `main()` is an infinite loop *by design*.

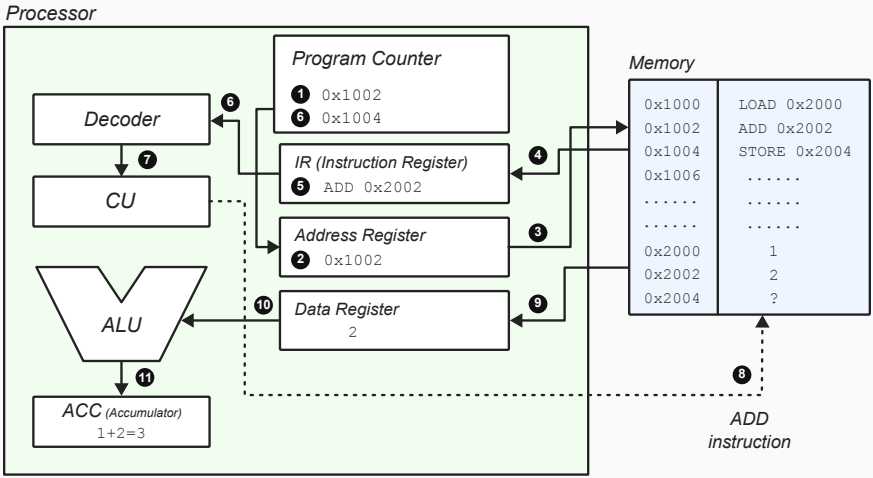
© Muním Zabidi

Executing LOAD 0x2000



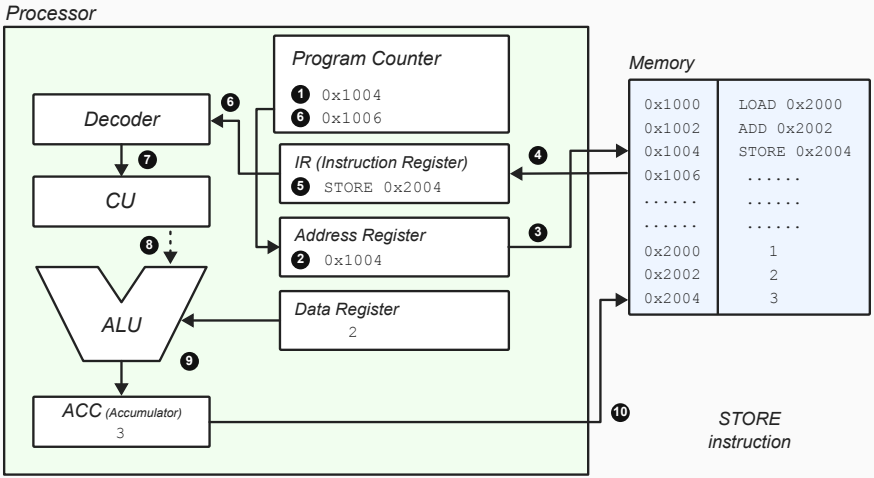
The value `1` at address `0x2000` reaches the Accumulator. This answers questions 1–3 of the trace. Note step **6**: the PC increases during decode, not after execute.

Executing ADD 0x2002



The value 2 is fetched, and the ALU adds it to the Accumulator. The Accumulator becomes 3. This answers questions 4 and 5.

Executing STORE 0x2004



The Accumulator's 3 is written out to 0x2004. This answers question 6. The program is now complete: three instructions, each made of several internal operations.

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Inside the Computer

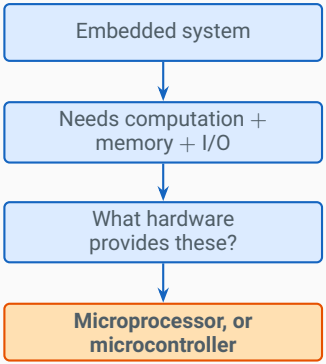
4 Buses and Memory-Mapped I/O

5 Inside the CPU

6 Execution Cycle and Program Trace

7 **Meet the Board**

8 From C to a Pin



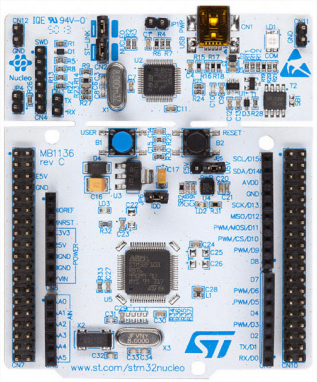
An embedded system is a *requirement*. Something must satisfy that requirement.

The question this raises

What kind of computer belongs inside an embedded system?

We will answer this question using one real board, so every idea in this chapter has a physical home.

© Muním Zabidi



STM32 Nucleo-F446RE

- **ST-LINK** debugger — the upper third of the board, past the perforation. One USB cable handles power, programming, debugging, and a virtual COM port.
- **USER button** (B1) on PC13
- **User LED** (LD2) on PA5

The inner headers follow the Arduino shield layout. The outer **morpho** headers bring out every remaining pin.

Every blink program in this book drives PA5. Everything we discuss from here on will eventually explain this board.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Inside the Computer

4 Buses and Memory-Mapped I/O

5 Inside the CPU

6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin

We Can Now Answer the Question We Started With

How does a program cause hardware to do something?

We just traced `z = x + y;` through LOAD, ADD, and STORE — ordinary memory addresses, nothing more. But the same load/store mechanism works on any address. Give it the address of a peripheral register instead of a memory cell, and it changes hardware, not a variable.

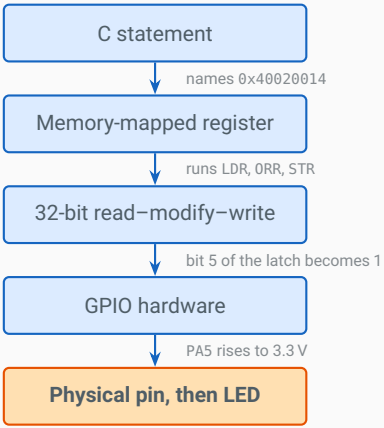
One Line of C, Back on the Board

Look at this line. It appears in almost every STM32 program ever written:

```
GPIOA->ODR |= (1U << 5);
```

You already know every symbol here: a pointer, a structure member, a compound bitwise-OR assignment, and a left shift.

What may not be obvious is that running this one statement turns on an LED: the LED on PA5, on the board you met at the start of this chapter.



© Muním Zabidi

The CPU still does not know what an LED is

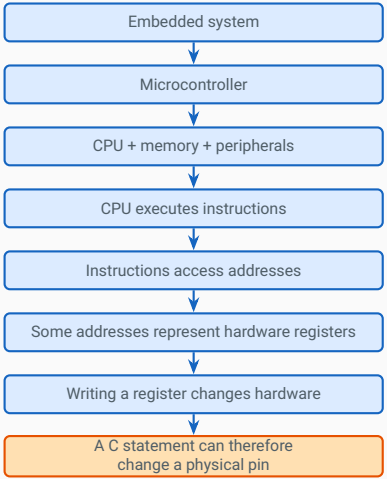
It only executes instructions and accesses addresses. Every step was ordinary: the same `LDR`/`ORR`/`STR` pattern as any other read-modify-write.

> Turning on an LED is writing a number to an address.

You asked this at the start of the chapter, as an unexplained claim. Now you can derive it: from a C statement, to a register write, to a voltage on a physical pin.

© Munim Zabidi

Summary: The Causal Chain



Terminology covered

- Embedded systems and SWaP-C
- Microprocessor vs microcontroller
- Buses
- CPU registers, ALU, control unit
- Fetch–decode–execute
- Memory-mapped I/O

Next: Chapter 2

Turns this generic CPU into the specific one on your bench.

© Muním Zabidi